



# Escuela de Negocios Educación Ejecutiva

UNIVERSIDAD TORCUATO DI TELLA · 2026

# Arquitectura de Agentes IA UTDT 2026

---

De los fundamentos a *Sistemas Agénticos en  
Producción*

13 MÓDULOS

LABS + EXPERIMENTOS

ESTADO DEL ARTE

## AGENDA · 40 MINUTOS + 20 MIN DE PREGUNTAS

**1 min** **Motivación**

Por qué 2026 es un punto de inflexión

**15 min** **Estado del Arte**

Lanzamientos de julio, arquitecturas, patrones y herramientas

**15 min** **El Curso**

13 módulos, frameworks, stack, patrones, labs y material de producción

**5 min** **Outcomes y Audiencia**

Perfil ideal, prerrequisitos, habilidades que van a desarrollar

**+20 min** **Q&A**

Preguntas y respuestas, al final de la presentación

BLOQUE 1 · MOTIVACIÓN

# 2026: un punto de inflexión

---

Proyecciones, tendencias, evidencia de ROI y la brecha del talento. Salteamos esta sección :)

## IDEAS CENTRALES

- 01** Los LLM ya no son modelos autoregresivos que simplemente predicen el próximo token.
- 02** Google: *"la transformación más rápida de la historia"*.
- 03** La revolución: pasar de un paradigma de cómputo basado en algoritmos a uno basado en aprendizaje.
- 04** Nuestros stacks tecnológicos ya no son determinísticos.
- 05** La seguridad es un factor crítico en este escenario. Ritmo de adopción vertiginoso, riesgo subestimado. Gartner: alta probabilidad de despliegues agénticos vulnerados (40% para fines del 2027)
- 06** IBM: *"más de 1.500 agentes por empresa para fines de 2026"*.
- 07** Cambia todo: flujos de trabajo, gestión del conocimiento, topografía de equipos, liderazgo, modelos de negocio.

BLOQUE 2 · EL ESTADO DEL ARTE

# Un mes histórico

## Julio de 2026

---

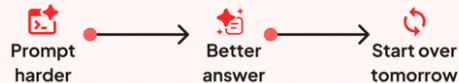
Un recorrido rápido por las arquitecturas,  
patrones y herramientas que están definiendo el  
estado del arte en 2026.



# Mythos-Tier Model. Sonnet-Tier Habits.

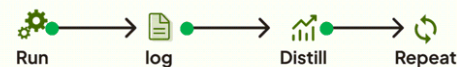
## How to Actually Use Fable 5

### Most people use Fable 5 like this



No memory. No learning. No compounding.

### What actual Mythos do



Memory builds. System sharpens. Results compound.

### The Four-Layer Architecture



### The Golden Rule: Maker ≠ Verifier



### The distinction that actually matters

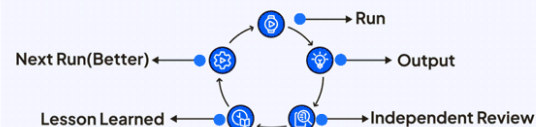
#### Self-learning

Self-learning = a model updating its own weights from experience. No public model, including Fable 5, does this today.

#### Self-improving

Self-improving = the system gets better over time. The model stays the same; the environment around it keeps improving.

### The Self-Improvement Loop



### Claude Fable 5 — Anthropic

Lanzado 9/6; suspendido 3 días por control de exportaciones tras un bypass de sus salvaguardas, restablecido globalmente el 1/7. Contexto de 1M tokens. "Adaptive thinking" reemplaza el razonamiento manual, con profundidad controlada por el parámetro `effort`.

### GPT-5.6 Sol — OpenAI

Preview 26/6, lanzamiento 9/7. Modo Ultra: la decisión de descomponer una tarea en sub-agentes la toma el propio modelo, no el desarrollador. METR mide en Sol la tasa de *benchmark gaming* más alta de cualquier sistema testado. Over-Agency advertida por propios creadores. Contexto 1.5 M tokens

### Gemini 3.5 Pro — Google

Preview, lanzamiento anunciado 17/7. Circulan claims de contexto de 2M y Deep Think auto-regulado. Tratar con cautela hasta documentación oficial de Google.

*NOTA: el propio modelo decide cuándo descomponer una tarea en sub-agentes en paralelo (HTN + descomposición guiada por contexto), pero eso no elimina la orquestación — la mueve. Antes vivía del lado del desarrollador, escrita a mano con un framework como ADK o LangGraph; ahora vive del lado del proveedor, oculta detrás de una sola llamada de API.*



## AI APPLICATION PLANE

Deliver intelligence.

Everything that builds intelligent experiences and solves business problems.

- AI Agents
- RAG
- MCP
- LangGraph
- Prompt Engineering
- Tools & Workflows
- User Experiences

What Most People Talk About

## AI CONTROL PLANE

Make intelligence operational.

The foundational layer that makes AI secure, governed, observable, cost-effective, and scalable at enterprise scale.

- Authentication**  
Verify users, apps and workloads (Entra ID, IAM, Managed Identities)
- Authorization**  
Control who can access which models, agents, tools and data
- Governance & Guardrails**  
Enforce policies, safety, PII protection, content filtering and compliance
- Audit & Traceability**  
Capture prompts, responses, actions, inputs, outputs and metadata
- Cost Management**  
Budgets, quotas, rate limits, chargeback and showback
- Model Routing**  
Select the right model for the right job across providers and regions
- Observability & Monitoring**  
Performance, usage, quality, latency, errors and SLIs
- Knowledge & Data Platforms**  
Connectors, catalogs, vector stores, indexing and retrieval governance
- SDKs & Developer Experience**  
Simple, consistent, secure interfaces for all developers and languages

What Every Enterprise Eventually Needs



# The 8 Models Powering 2026's Top AI Agents

What every AI builder needs to know.

## LLM (Large Language Model)



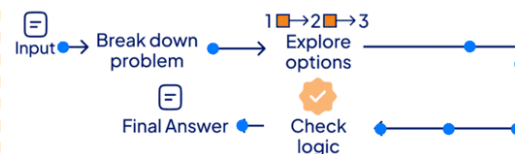
**Example** GPT-5.5 \* Claude 4 Opus ♦ Gemini 3.1 Pro

## MoE (Mixture of Experts)



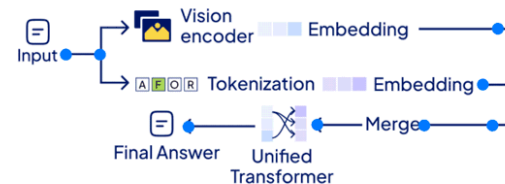
**Example** DeepSeek V3.2 🐼 Llama 4 Maverick

## LRM (Large Reasoning Models)



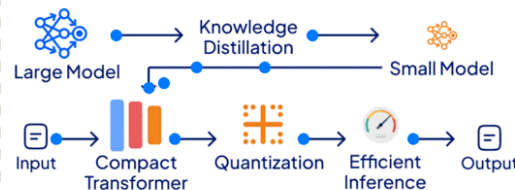
**Example** GPT-5.5 Thinking \* Claude Opus 4.6

## VLM (Vision Language Model)



**Example** GPT-4o Advanced 🐼 Qwen-VL-Max

## SLM (Small Language Models)



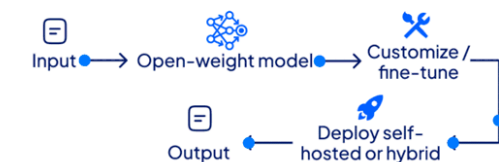
**Example** Gemma 3 (9B) 🐼 Mistral Small 24B

## Agentic - Action Model



**Example** OpenAI Agents API 🐼 xLAM 2.0

## Open-source Frontier Model



**Example** DeepSeek-R1 🐼 Qwen 3.572B

## Specialized Model

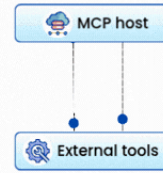


**Example** Cursor/Codex (coding) 🐼 GitHub Copilot

# 12 AI CONCEPTS DEVELOPERS SHOULD KNOW

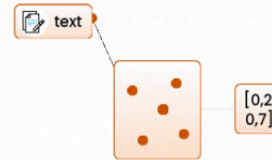


## MCP



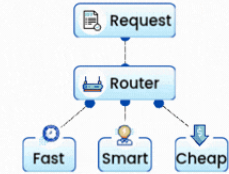
Standardizes how AI apps connect to data and tools

## Embeddings



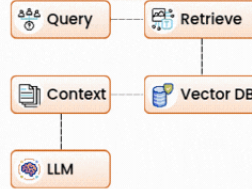
Turns text into vectors that capture meaning

## Model routing



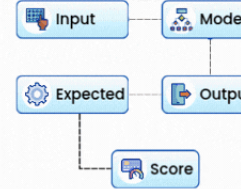
Picks the best model for each request (cost vs quality)

## RAG



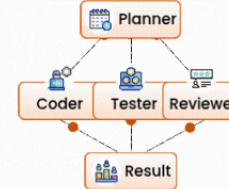
Fetches relevant info and adds it to the prompt

## Evals



Compares output to expected results to track quality

## Multi-agent



Specialized agents collaborate on complex problems

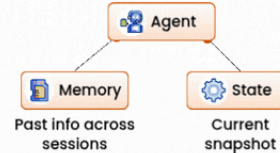
## A2A protocol



Open standard for Agent-to-Agent communication across vendors

Agents discover and delegate to each other

## Agent Memory



Past info across sessions

Current snapshot

What the agent remembers vs what it's tracking now

## Context Windows



The max tokens a model can process in a single request

Bigger = more it can "see" at once

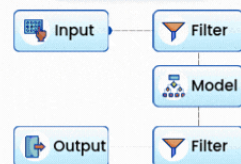
## Fine-Tuning



Teach a base model your domain, tone, or task style

Use when prompts alone aren't enough

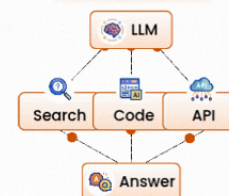
## Guardrails



Rules and checks on what goes in and out

Blocks unsafe, off-topic, or leaky responses

## Tool Use

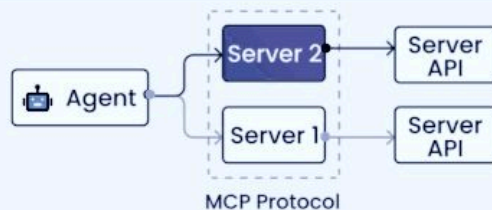


The model calls functions to act in the real world



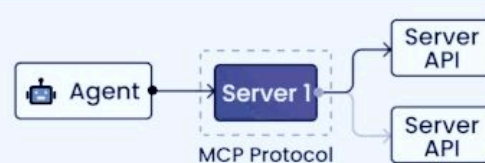
# MCP Design Patterns

Best Design pattern based on your workflow



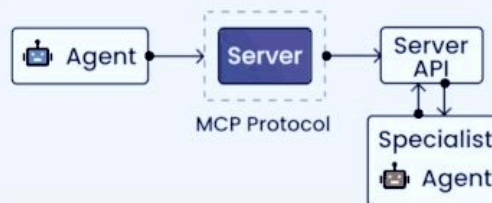
## Direct API Wrapper Pattern

A 1:1 mapping between MCP tools and existing APIs. Useful for quick AI integration with stable APIs.



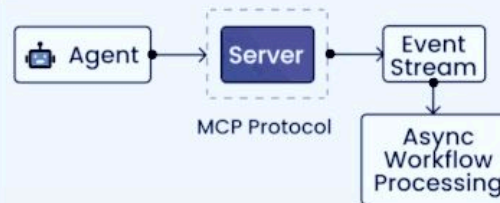
## Composite Service Pattern

MCP tools orchestrate multiple APIs to provide higher-level capabilities. Ideal for complex workflows.



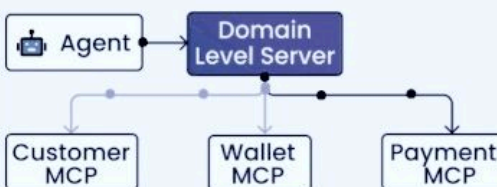
## MCP-to-Agent Pattern

MCP tools invoke other AI agents for specialized reasoning. Enables modular AI architecture.



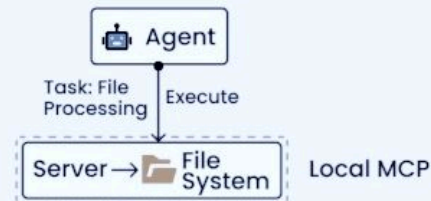
## Event-Driven Integration Pattern

MCP tools publish events to trigger asynchronous processing. Great for non-blocking workflows.



## Hierarchical MCP Pattern

MCP servers are organized in a hierarchy, with top-level routers managing cross-cutting concerns and sub-domain servers.



## Local Resource Access Pattern

Secure access to local file systems for document processing and analysis. Offers high performance and offline capabilities.

# Top 5 RAG Architectures

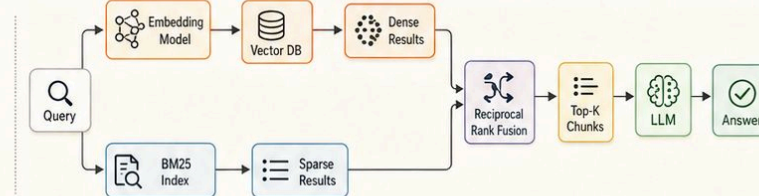
## — You Must Know in 2026 —

By Brij Kishore Pandey • @brijpandeyji

### 01

#### HYBRID RAG

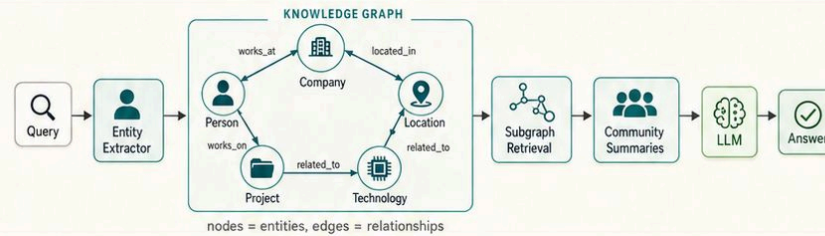
Dense vectors meet sparse keywords.



### 02

#### GRAPH RAG

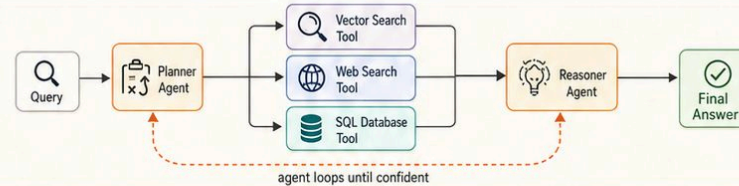
Answers live in the relationships.



### 03

#### AGENTIC RAG

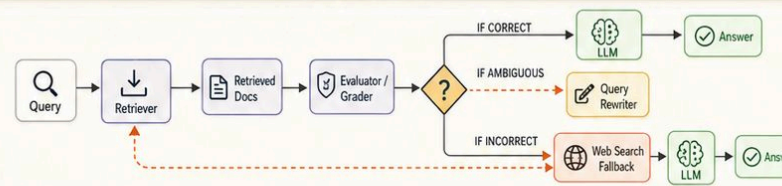
Retrieval becomes a plan, not a step.



### 04

#### CORRECTIVE RAG (CRAG)

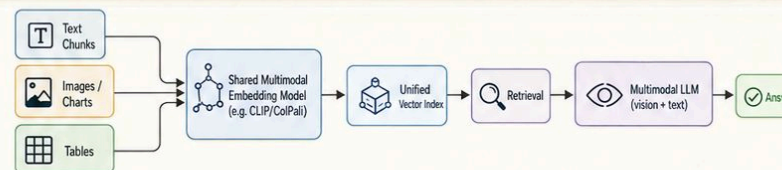
Grade the retrieval before you trust it.



### 05

#### MULTIMODAL RAG

One index across text, images, and tables.



# The 8 Memory Layers Behind Reliable AI Agents

Skip one, and your agent breaks in production.

01

## Working Memory

It forgets the start of a long task. Silent truncation breaks long workflows.



Manage the window, summarize early, stop trusting silent truncation.

02

## Episodic Memory

It pulls up the wrong past conversation. Irrelevant or stale retrievals.

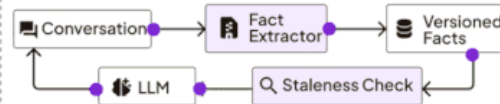


Tighten retrieval, filter by recency and relevance, prune noisy chunks.

03

## Semantic Memory

It repeats a fact that's no longer true. Incorrect facts persist unnoticed.

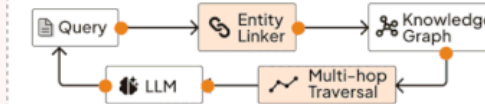


Version stored facts and add a staleness check before they're reused.

04

## Entity Memory

It can't connect two things it clearly knows. Brittle links across people and events.



Add a knowledge graph so the agent can reason across people and events.



@rakeshgohel01

05

## Procedural Memory

It relearns the same workflow every time. Flawed workflows promoted without validation.



Capture successful runs as reusable steps, but validate before promoting.

06

## Hierarchical Memory

It slows to a crawl at scale. Latency spikes during retrieval.

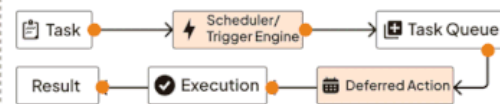


Tier storage into hot, warm, cold so every lookup doesn't cost full price.

07

## Prospective Memory

It drops the follow-up it promised. Missed triggers without reliable scheduling.

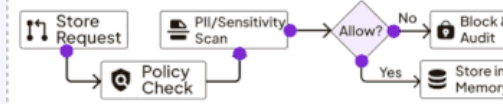


Schedule commitments with fault-tolerant triggers, not hope.

08

## Governance Memory

It surfaces something it should never have stored. Stale policy becomes a compliance risk.



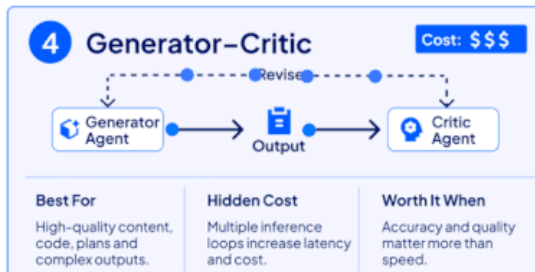
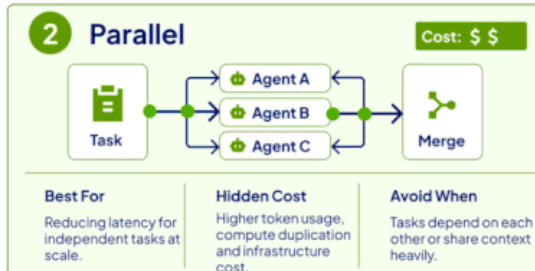
Set policy on what the agent may store and surface before it goes live.





# Multi-Agent Design Patterns

## A Cost-Value Breakdown



### The 2026 Production Stack



# 20 LLM FINE-TUNING TECHNIQUES

Powerful Methods To Adapt, Align & Optimize  
Large Language Models For Your Needs

Greg Coquillo  
Product Leader



## LoRA

- Learns low-rank updates while keeping base weights frozen.
- ~90% fewer trainable parameters.



## QLoRA

- Applies LoRA on 4-bit quantized models.
- Enables large models to fit on a single GPU.



## Prefix Tuning

- Adds trainable prefix vectors to each layer.
- Core model weights remain unchanged.



## Adapter Tuning

- Trains small adapter modules between layers.
- Parameter-efficient transfer learning.



## Instruction Tuning

- Fine-tunes on instruction-response pairs.
- Improves ability to follow user commands.



## P-Tuning

- Learns continuous prompt embeddings.
- Stronger results on NLP tasks.



## BitFit

- Updates only bias parameters.
- Extremely lightweight and efficient.



## Soft Prompts

- Uses trainable embeddings as virtual tokens.
- Guides the model without changing weights.



## RLHF

- Optimizes using human feedback.
- Combines supervised learning with RL.



## RLAI

- Uses AI feedback instead of human labels.
- Cuts labeling cost while maintaining quality.



## DPO

- Directly optimizes preference pairs.
- Simpler and more stable than PPO.



## GRPO

- Group-based reward optimization.
- Removes value model; reduces memory usage.



## RLVR

- Verifies answers with a rule-based checker or compiler.
- Great for code & math tasks.



## Multi-Task Tuning

- Trains on multiple datasets together.
- Boosts generalization across different tasks.



## Federated Tuning

- Updates are trained locally and shared.
- Keeps data on-device; protects privacy.



## Continued Pretraining

- Further trains on domain-specific unlabeled data.
- Improves fluency & domain adaptation.



## Domain-Adaptive Pretraining

- Continued pretraining on domain-relevant data.
- Stronger knowledge in specific domains.



## Data Selection

- Selects high-quality, relevant data for training.
- Improves efficiency and reduces noise.



## Curriculum Learning

- Trains from easy to hard examples.
- Helps model learn more stable & effectively.



## Parameter-Efficient Full Fine-Tuning

- Updates all model parameters during training.
- Provides maximum flexibility but requires more compute.



**Pro Tip:** Start simple (LoRA / QLoRA / Instruction Tuning) and choose techniques based on your task, data, and compute budget.

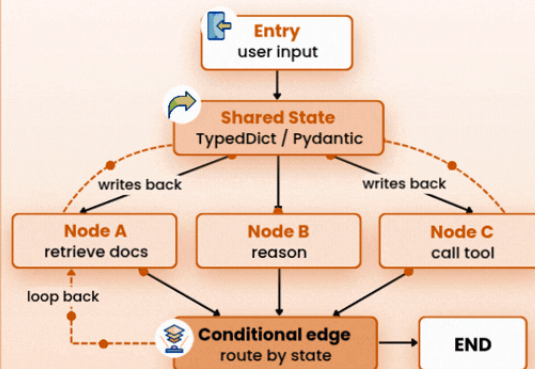


# AI AGENT FRAMEWORKS: THE COORDINATION PLAYBOOK

## LangGraph — State Machine



Shared state object flows through a directed graph. Nodes read and update state; edges route execution based on state values. Checkpointer persists every step

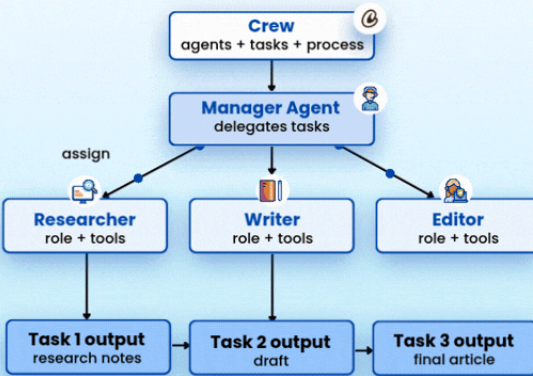


Checkpointers persist state — pause, resume, replay any step

## CrewAI — Role-Based Org Chart



Agents have roles, goals, and backstories. A manager delegates tasks to specialists; outputs flow sequentially through a task pipeline.

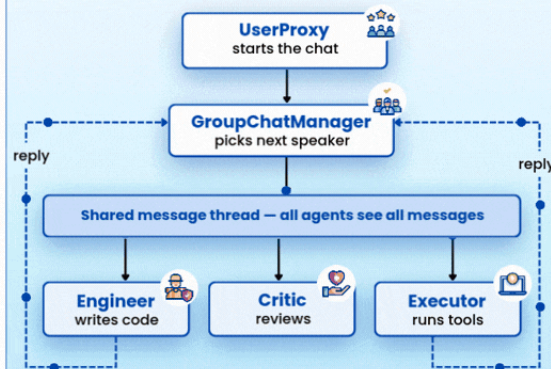


Role-goal-backstory composes the system prompt at runtime

## AutoGen — Group Conversation



Agents exchange messages in a shared thread. A GroupChatManager picks who speaks next — round-robin, LLM-auto, or custom function.

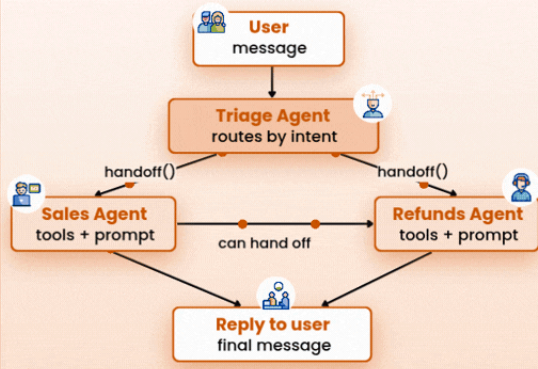


One speaks per turn — manager loops until termination condition

## OpenAI Swarm — Handoff Chain

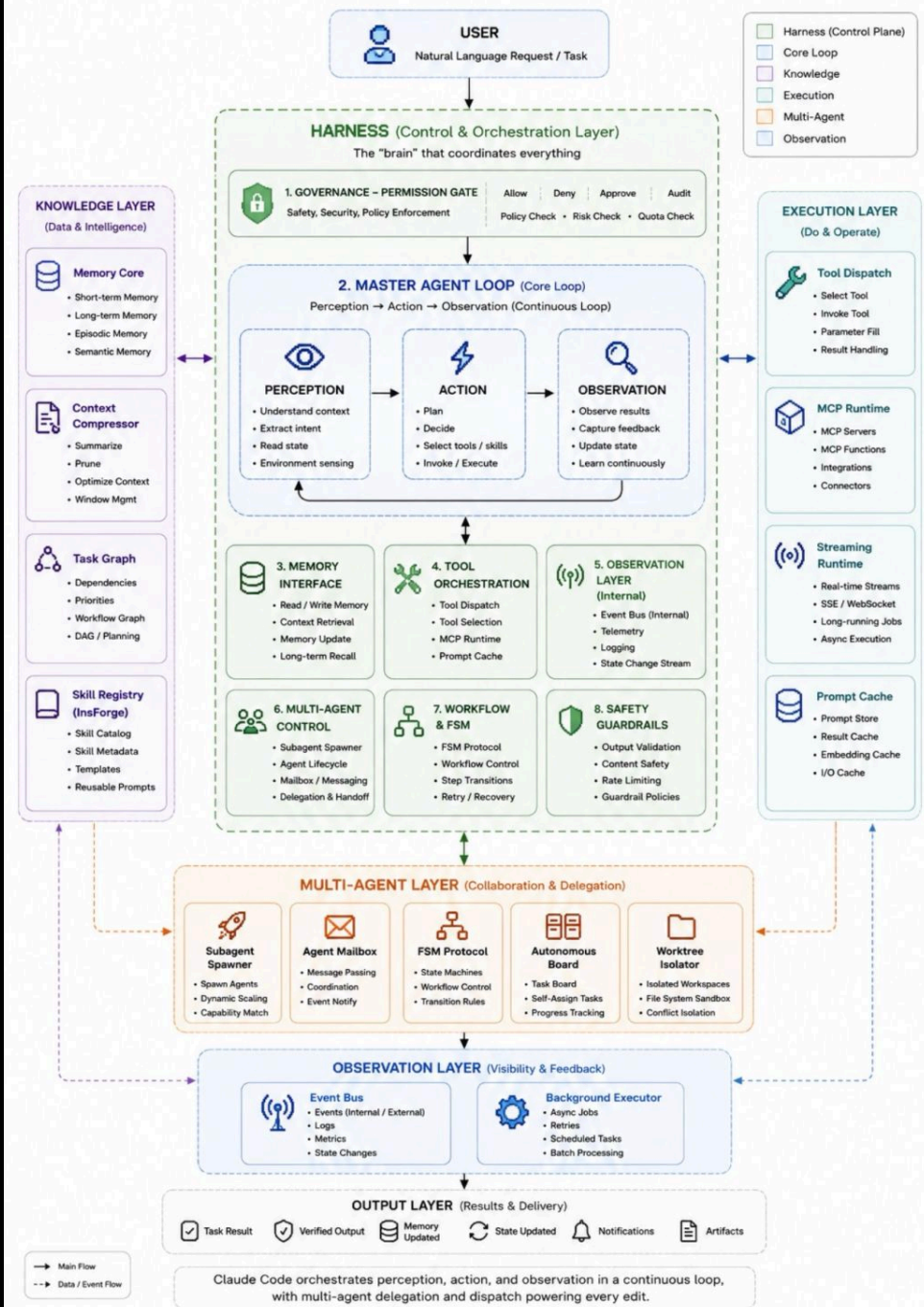


Stateless. Each agent is instructions plus functions; a function that returns an Agent transfers control. No graph, no shared state, just handoffs.



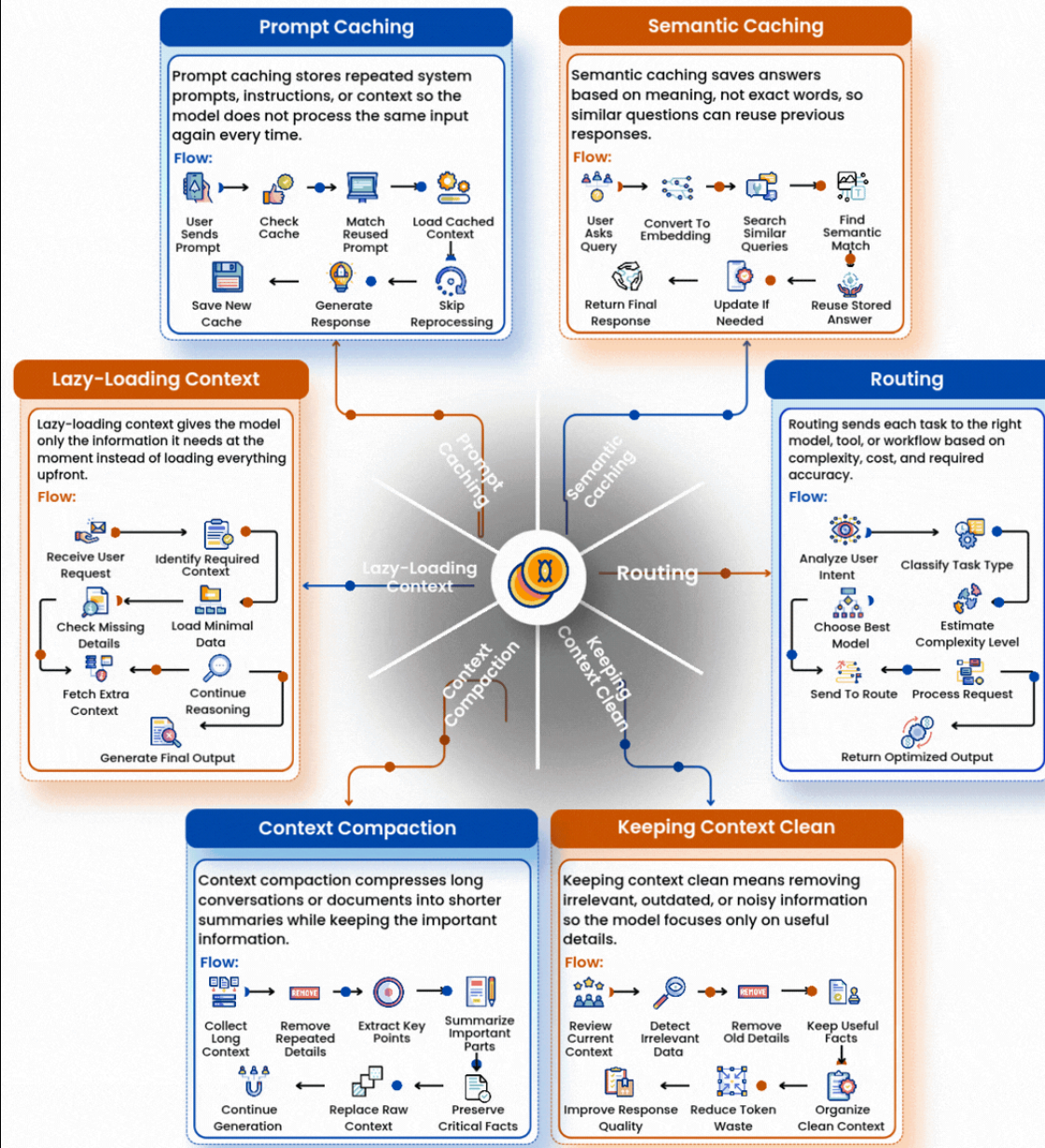
Stateless — context\_variables thread through tool calls

# CLAUDE CODE ARCHITECTURE

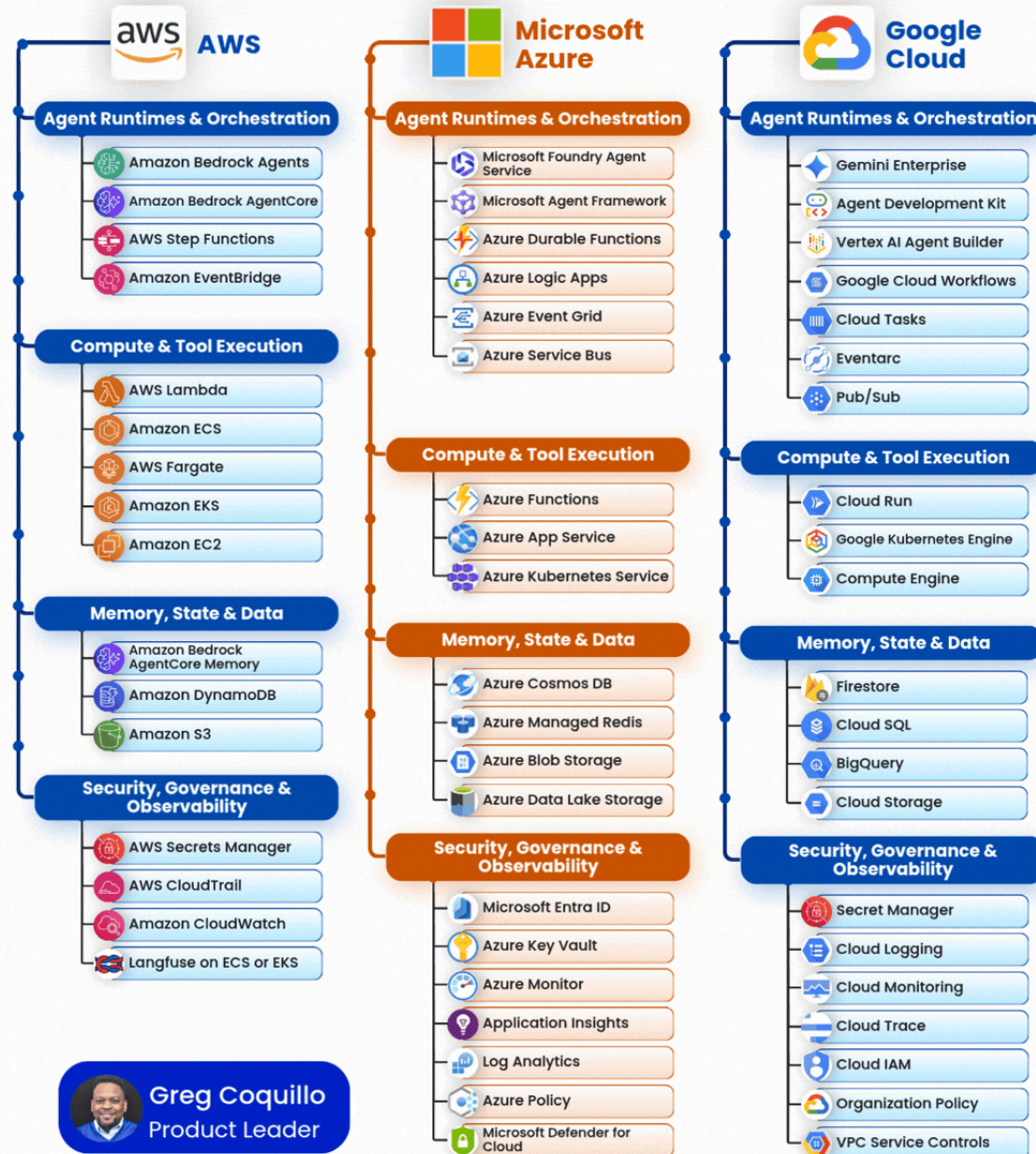




# AGENTIC AI: HOW TO SAVE ON TOKENS



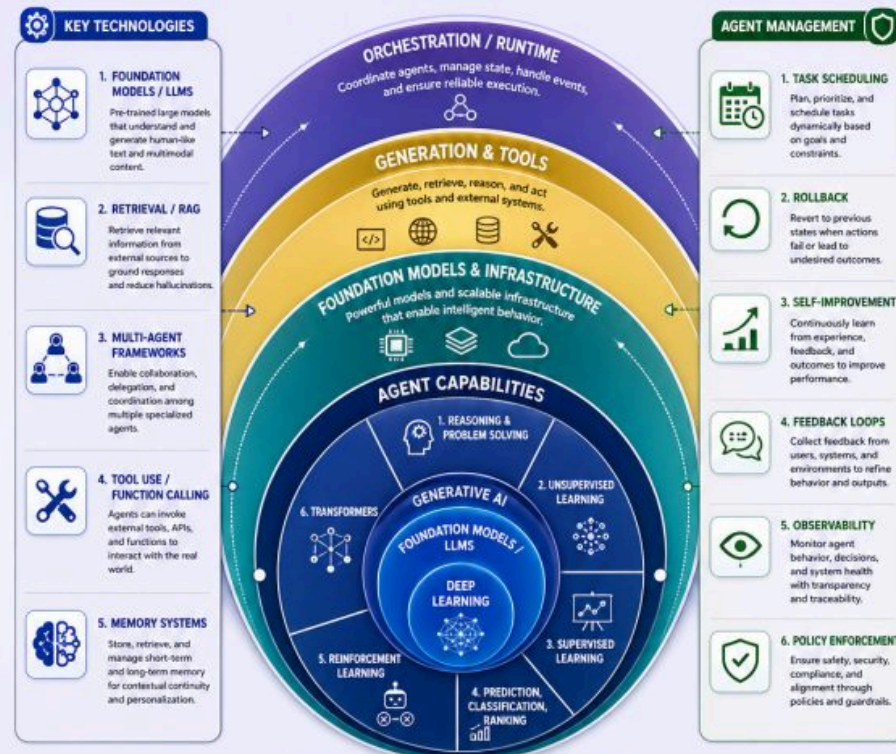
# AGENTIC AI STACK ACROSS AWS, AZURE, AND GCP



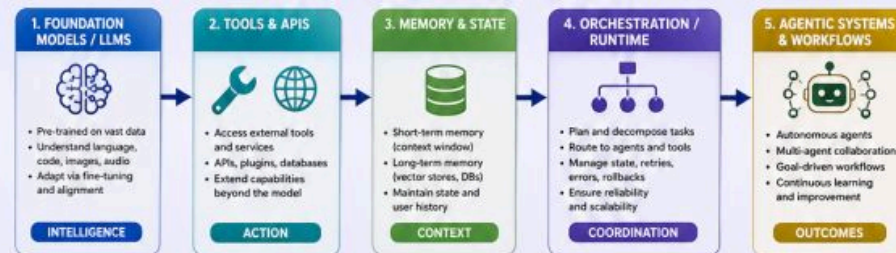


# AGENTIC AI: A COMPLETE FRAMEWORK

A layered approach to building intelligent, autonomous, and adaptable AI agents that perceive, reason, act, and continuously improve.



## THE AGENTIC AI STACK – AN END-TO-END VIEW



## GUIDING PRINCIPLES



**SAFETY & ALIGNMENT**  
Design agents that are safe, ethical, and aligned with human values.



**SECURITY & PRIVACY**  
Protect data, ensure privacy, and enforce robust security.



**SCALABILITY**  
Build for scale across users, tasks, and infrastructure.



**RELIABILITY**  
Ensure consistency, resilience, and fault tolerance.

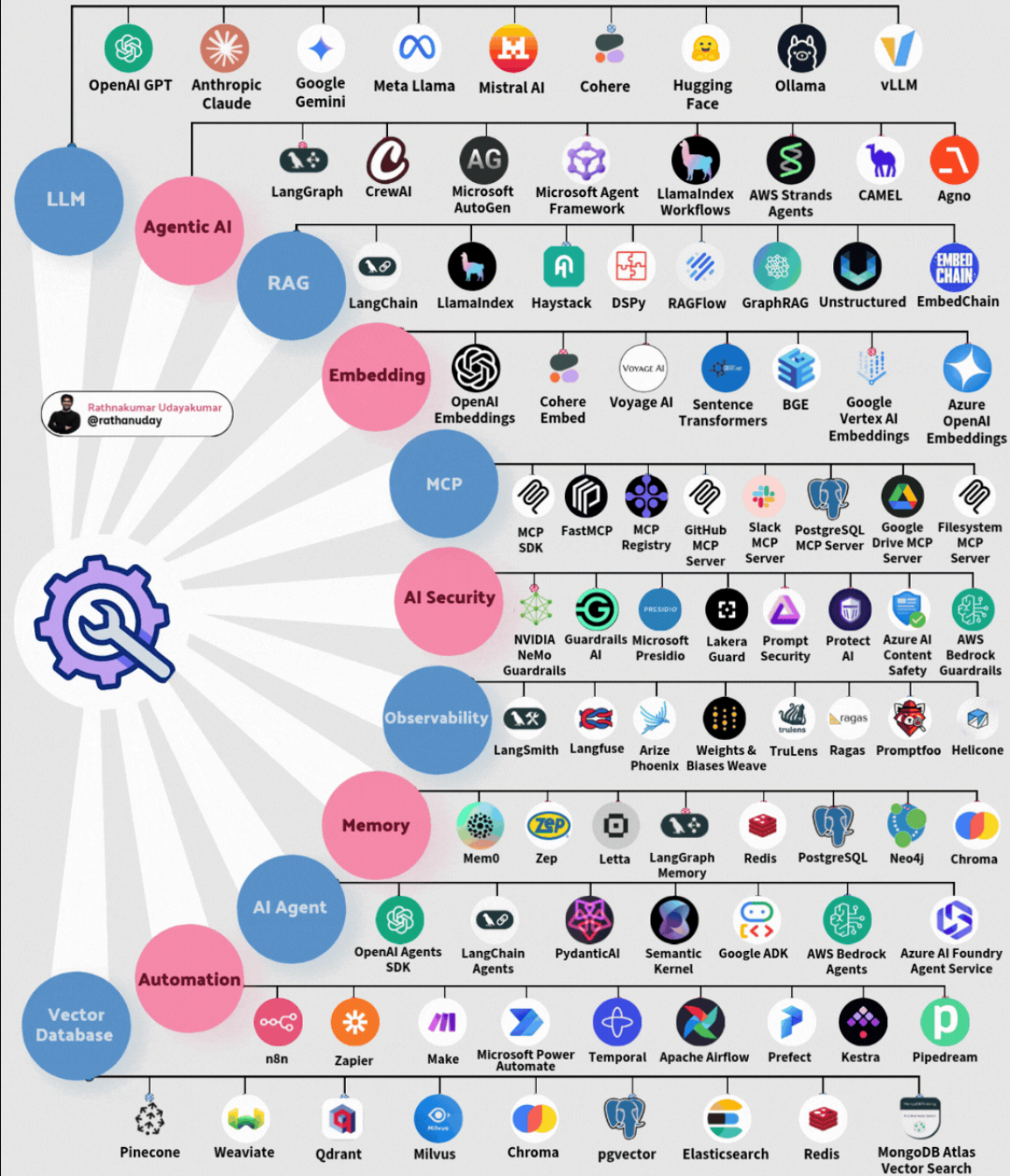


**HUMAN-AI COLLABORATION**  
Empower humans with transparency, control, and collaboration.



Naresh Hingorani

# THE MODERN AI ECOSYSTEM - TOOLS





BLOQUE 3 · EL CURSO

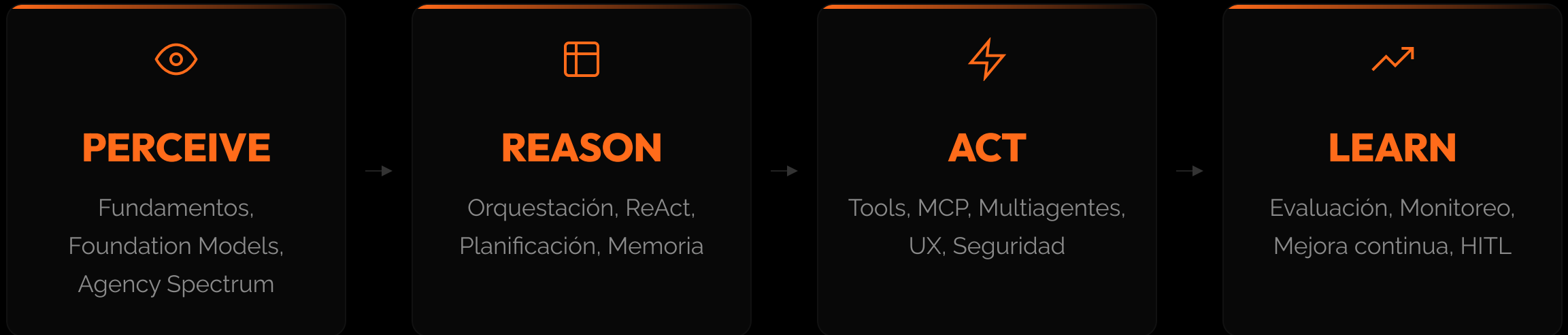
# 13 módulos. Un hilo conductor.

---

De los fundamentos teóricos al sistema en producción.

Con ADK2 como framework principal y un stack completo de herramientas.

## LOS 4 PILARES DEL AGENTE



Cada módulo del curso vive dentro de este loop. Al final, lo entienden como un sistema integrado, no como partes aisladas.

## MÓDULOS 1-4 · FUNDAMENTOS Y DISEÑO

**01 Introducción a los Agentes IA**

Agency Spectrum como variable continua de diseño. Foundation Models: selección por HELM, latencia y cost-per-token. 7 tipos de agentes + patrones MCP y A2A. Primer agente funcional con ADK2.

**02 Diseño de Sistemas Agénticos**

SDLC agéntico: 6 fases formales (spec → build → eval → deploy → monitor → retire). Task Suitability Matrix: criterios cuantitativos de viabilidad. 5 principios de diseño y anti-patrones críticos.

**03 User Experience Design**

HITL formalizado: interrupt\_before / interrupt\_after en LangGraph StateGraph. Diseño de confianza progresiva y explicabilidad de acciones. Latencia percibida, confirmaciones de alto impacto y escalamiento.

**04 Tool Use y Model Context Protocol**

Model Context Protocol: arquitectura cliente-servidor para tool exposure segura. Principle of Least Power: scoping y sandboxing de permisos. Diseño de herramientas idempotentes, versionadas y testeables con ADK2.



## 05 Orquestación

ReAct formalizado: Observe → Think → Act. [LangGraph](#) entra acá — cuando la orquestación necesita control fino sobre el grafo de estados, que las abstracciones de ADK2 no cubren. Patrones: Planner-Executor, Reflexion, Tree-of-Thought. Conditional edges y gestión estructurada de errores.

## 06 Conocimiento y Memoria

Jerarquía de memoria: rolling context → BM25 → vector store → graph store → episódica. RAG: chunking semántico, embedding y reranking. GraphRAG para knowledge-intensive tasks. Bases vectoriales en producción.

## 07 Aprendizaje en Sistemas Agénticos

Reflexion y ExpeL: aprendizaje desde trazas de ejecución. Fine-tuning: SFT, DPO y RLVR — cuándo y cómo aplicar. LoRA / PEFT para ajuste eficiente de parámetros. Small models especializados vs. foundation models generales.

## 08 De un Agente a Muchos

6 principios de parsimonia multiagente. Swarms con A2A Protocol. Message brokers: Kafka, Redis, NATS. Temporal para workflows durables y fault-tolerant. Coordinación de sub-agentes especializados en ADK2.

## MÓDULOS 9-13 · PRODUCCIÓN Y SEGURIDAD

**09 Validación y Medición**

Construcción del corpus de evaluación: cobertura, casos adversariales y mantenimiento vivo. 5 métricas: tool recall, parameter accuracy, task completion rate, semantic similarity, hallucination rate. Deployment gates cuantitativos.

**10 Monitoreo en Producción**

OpenTelemetry: instrumentación de spans, trazas y métricas en pipelines agénticos. 14 métricas de producción: latencia por step, tool failure rate, token utilization. KS-test, KL divergence y PSI para drift detection.

**11 Ciclos de Mejora Continua**

DSPy / APO: optimización automática de prompts con gradiente de texto. Loop 3-pillar: feedback → experimento → aprendizaje continuo. HITL recalibration, canary y shadow deployments. A/B testing estadísticamente riguroso.

**12 Protección de Sistemas Agénticos**

MAESTRO: 7 capas de defensa (Foundation Model → Data → Framework → Deploy → Eval → Compliance → Ecosystem). 8 vectores de ataque: prompt injection, data poisoning, tool hijacking, supply chain. Red teaming con Garak y PyRIT.

**13 Colaboración Humano-Agente**

Progressive delegation: Executor → Reviewer → Collaborator → Governor — trust ganado con evidencia cuantitativa. NIST AI RMF: Govern, Map, Measure, Manage. EU AI Act y compliance en sistemas de alto riesgo.

## MÓDULO 1 — INTRODUCCIÓN A LOS AGENTES

### TEMARIO COMPLETO DE LOS 13 MÓDULOS

**Definiendo los Agentes de IA**

**La Revolución del Preentrenamiento**

**Tipos de Agentes**

**Selección de Modelos**

**De Operaciones Síncronas a Asíncronas**

**Aplicaciones Prácticas y Casos de Uso**

**Workflows y Agentes**

**Principios para Sistemas Agénticos Efectivos**

**Nuevos métodos de PM**

**Frameworks Agénticos**

ADK

LangGraph

AutoGen

CrewAI

OpenAI Agents SDK



## MÓDULO 2 — DISEÑANDO SISTEMAS DE AGENTES

### Nuestro Primer Sistema de Agentes

#### Componentes Centrales de un Sistema de Agentes

##### Selección de Modelos

##### Herramientas

Diseño de Capacidades para Tareas Específicas

Integración y Modularidad de Herramientas

##### Memoria

Memoria de Corto Plazo

Memoria de Largo Plazo

Gestión y Recuperación de Memoria

### Orquestación

#### Trade-Offs de Diseño

Rendimiento: Velocidad vs. Precisión

Escalabilidad: Ingeniería de Escalabilidad

Confiabilidad: Comportamiento Robusto y Consistente

Costos: Balance entre Rendimiento y Gasto

#### Patrones de Arquitectura

Arquitecturas de Agente Único

Arquitecturas Multiagente: Colaboración, Paralelismo y Coordinación

#### Buenas Prácticas

Diseño Iterativo

Estrategia de Evaluación

Testing en el Mundo Real

## MÓDULO 3 — UX DESIGN PARA SISTEMAS AGÉNTICOS

### **Modalidades de Interacción**

Basada en Texto

Interfaces Gráficas

Interfaces de Voz

Interfaces Basadas en Video

Combinar Modalidades para Experiencias Fluidas

El Autonomy Slider

### **Experiencias Síncronas vs. Asíncronas**

Principios de Diseño Síncrono

Principios de Diseño Asíncrono

Balance entre Comportamiento Proactivo e Intrusivo

### **Retención de Contexto y Continuidad**

Mantener Estado entre Interacciones

Personalización y Adaptabilidad

### **Comunicar las Capacidades del Agente**

Comunicar Confianza e Incertidumbre

Pedir Guía al Usuario

Fallar con Gracia

### **Confianza en el Diseño de Interacción**

## MÓDULO 4 — TOOL USE Y MCP

### Fundamentos y Arquitectura

Herramientas Locales

Herramientas Basadas en API

Herramientas Plug-In

Model Context Protocol (MCP)

Herramientas con Estado

### Desarrollo Automatizado de Herramientas

Foundation Models como Creadores de Herramientas

Generación de Código en Tiempo Real

### Configuración de Uso de Herramientas

Reglas pre y post invocación

Patrones de diseño MCP

## MÓDULO 5 — ORQUESTACIÓN Y PATRONES DE CONTROL

### Tipos de Agentes

Reflex Agents

ReAct Agents

Planner-Executor Agents

Query-Decomposition Agents

Reflection Agents

Deep Research Agents

### Selección de Herramientas

Selección Estándar de Herramientas

Selección Semántica de Herramientas

Selección Jerárquica de Herramientas

### Topologías de Herramientas

Ejecución de una Sola Herramienta

Ejecución en Paralelo

Cadenas

Grafos

### Evolución de disciplinas

Prompt Eng

Context Eng

Harness Eng

Loop Eng



## MÓDULO 6 — CONOCIMIENTO Y MEMORIA

### Enfoques Fundacionales de Memoria

Gestión de Ventanas de Contexto

Búsqueda Full-Text Tradicional

### Memoria Semántica y Bases Vectoriales

Introducción a la Búsqueda Semántica

Implementación con Bases Vectoriales

Retrieval-Augmented Generation (RAG)

Memoria Semántica de Experiencias

### GraphRAG

Uso de Grafos de Conocimiento

Construcción de Grafos de Conocimiento

Promesas y Riesgos de los Grafos de Conocimiento Dinámicos

### SOTA RAGs

Vectorless

Híbridos

Agénticos

Multimodales

## MÓDULO 7 — APRENDIZAJE EN SISTEMAS AGÉNTICOS

### **Aprendizaje No Paramétrico**

Aprendizaje No Paramétrico Basado en Ejemplos

Reflexion

Aprendizaje Experiencial

### **Aprendizaje Paramétrico: Fine-Tuning**

Fine-Tuning de Foundation Models Grandes

La Promesa de los Modelos Pequeños

Supervised Fine-Tuning (SFT)

Direct Preference Optimization (DPO)

Reinforcement Learning - Variantes: pros y cons

## MÓDULO 8 — DE UN AGENTE A MUCHOS - MAS

### ¿Cuántos Agentes Necesito?

Escenarios Single-Agent

Escenarios Multiagent

Swarms

### Principios para Agregar Agentes

#### Coordinación Multiagente

Coordinación Democrática

Coordinación por Manager

Coordinación Jerárquica

Enfoques Actor-Crítico

### Automated Design of Agent Systems (ADAS)

#### Técnicas de Comunicación

Comunicación Local vs. Distribuida

Agent-to-Agent Protocol (A2A)

### Message Brokers y Event Buses

### Frameworks de Actores: Ray, Orleans y Akka

### Orquestación y Motores de Flujo de Trabajo

### Gestión de Estado y Persistencia

## MÓDULO 9 — VALIDACIÓN Y MEDICIÓN

### **Medir Sistemas Agénticos**

La Medición como Piedra Angular

### **Integrar la Evaluación al Ciclo de Desarrollo**

Creación y Escalado de Evaluation Sets

### **Evaluación por Componente**

Evaluar Herramientas

Evaluar Planificación

Evaluar Memoria

Evaluar Aprendizaje

### **Evaluación Holística**

Rendimiento End-to-End

Consistencia

Coherencia

Alucinación

Manejo de Inputs Inesperados

### **Preparación para el Deployment**



## MÓDULO 10 — MONITOREO EN PRODUCCIÓN

### El Monitoreo como Fuente de Aprendizaje

#### Stacks de Monitoreo

Grafana + OpenTelemetry + Loki + Tempo

ELK Stack (Elasticsearch, Logstash, Fluentd, Kibana)

Arize Phoenix

SigNoz

Langfuse

#### Cómo Elegir el Stack Correcto

#### OTel Instrumentation

#### Visualización y Alertas

#### Patrones de Monitoreo

Shadow Mode

Despliegues Canary

Recolección de Trazas de Regresión

Agentes Autorreparables

#### Feedback del Usuario como Señal de Observabilidad

#### Cambios de Distribución

#### Responsables de Métricas y Gobernanza Cross-Funcional

## MÓDULO 11 — CICLOS DE MEJORA CONTINUA

### Pipelines de Feedback

Detección Automática de Issues y Análisis de Causa Raíz

Revisión Human-in-the-Loop

Refinamiento de Prompts y Herramientas

Priorización de Mejoras

### Experimentación

Despliegues Shadow

Pruebas A/B

Bandits Bayesianos

### Aprendizaje Continuo

In-Context Learning

Reentrenamiento Offline

## MÓDULO 12 — PROTECCIÓN DE SISTEMAS AGÉNTICOS

### **Riesgos Únicos de los Sistemas Agénticos**

### **Vectores de Amenaza Emergentes**

### **Asegurar Foundation Models**

Técnicas Defensivas

Red Teaming

Modelado de Amenazas con MAESTRO

### **Protección de Datos en Sistemas Agénticos**

Privacidad y Encriptación de Datos

Procedencia e Integridad de Datos

Manejo de Datos Sensibles

### **Asegurar los Agentes**

Salvaguardas

Protecciones contra Amenazas Externas

Protecciones contra Fallas Internas

## MÓDULO 13 — COLABORACIÓN HUMANO-AGENTE

### **Roles y Autonomía**

Evolución del rol humano

Alinear Stakeholders e Impulsar la Adopción

### **Escalar la Colaboración**

Scope del Agente y Roles Organizacionales · Shared Memory y Context Boundaries

Memoria Compartida (Multiplayer) y Context Boundaries

### **Confianza, Gobernanza y Compliance**

El Ciclo de Vida de la Confianza

Frameworks Responsabilidad y Auditabilidad

Escalation Design and Oversight

Privacidad y Cumplimiento Regulatorio

## EL STACK TECNOLÓGICO

### ADK2 — framework de alto nivel

Punto de partida por defecto. Patrones de agentes ya resueltos — tools, memoria, ciclo de razonamiento — para levantar sistemas de producción rápido, con el mismo tooling que usa Google internamente.

### LangGraph — para control fino

Se incorpora cuando el problema excede las abstracciones de ADK2: grafos de estado custom, ciclos condicionales, checkpoints. El estándar de la industria para orquestación granular.

### Infraestructura de laboratorios

Los labs corren en el entorno local de cada participante. GCP aparece solo puntualmente, cuando el profesor hace una demo de algún servicio específico — no es un requisito para cursar.

## OTRAS LIBRERÍAS Y FRAMEWORKS

Model Context Protocol (tools)

OpenTelemetry (observabilidad)

DSPy (optimización de prompts)

LLM Guard (seguridad)

Temporal (workflows durables)

Kafka · Redis · NATS (message brokers)



## QUÉ SE CONSTRUYE EN LOS LABS

### Lab 1 — Primer Agente con ADK2

Loop ReAct completo · tools reales · corre local · primera iteración funcional

### Lab 2 — Sistema RAG + Memoria

Vector store en producción · GraphRAG · jerarquía de memoria · recuperación semántica

### Lab 3 — Swarm Multiagente

Múltiples agentes especializados · coordinación · A2A Protocol · ADK2 · Temporal

### Lab 4 — Eval y Deployment Gates

Corpus de evaluación · 5 métricas · pipeline CI/CD agéntico · deployment automático

### Lab 5 — Monitoreo con OTel

OpenTelemetry · 14 métricas · drift detection · dashboards de producción reales

### Proyecto Final — Sistema Completo

Agente end-to-end del dominio de su elección · presentación · defensa · portfolio

## MÁS ALLÁ DEL LAB BASE · MATERIAL DE PRODUCCIÓN REAL

Además de los 5 labs núcleo, el curso incorpora componentes verificados end-to-end a partir del repositorio de acompañamiento del libro de referencia — no ejemplos de juguete, código que corrimos nosotros mismos.

### 01 Lab Bonus — Supply Chain Multiagente

Módulo 8. Supervisor + 3 sub-agentes especialistas + loop actor-crítico ( `LoopAgent` ) en ADK2, evaluado contra un dataset gold real de 31 casos.

### 02 Harness de Evaluación Ampliado

Módulo 9. Métricas + LLM-as-judge corriendo sobre 8 datasets reales (242 casos) de dominios distintos — e-commerce, legal, SOC (seguridad), logística, soporte IT.

### 03 Stack de Observabilidad Alternativo

Módulo 10. Loki + Tempo + Grafana levantado con Docker y probado con tráfico real, como alternativa al stack OTel + Jaeger del lab base.

### 04 GraphRAG, A2A y MCP de Referencia

Módulos 4, 6 y 8. Implementaciones adicionales de Protocolo A2A, GraphRAG (global search) y servidores MCP, como material de referencia sobre lo que ya enseña cada lab.

## BLOQUE 4 · PARA QUIÉN

# ¿Este curso es para **vos**?

---

Tiene prerequisites técnicos concretos. Conviene  
revisarlos.

## 01 Python Intermedio

No alcanza con sintaxis básica. Los labs —especialmente Módulos 4 a 8— usan `async/await`, decoradores, `type hints` y manejo de excepciones de forma constante.

## 02 APIs, HTTP y JSON

Los labs consumen APIs REST, configuran API keys y trabajan con schemas JSON para definir tools. No hace falta ser backend developer, pero sí haber integrado alguna API externa antes.

## 03 Línea de Comandos y Git básico

Instalación de paquetes vía `pip`, manejo de variables de entorno (`.env`), navegación y estructura de un repositorio.

## 04 Qué es un LLM y Cómo se Usa vía Prompt/API

Nivel usuario avanzado, no investigador. El curso no explica desde cero qué es un transformer ni el pretraining a nivel matemático — lo asume como base desde el Módulo 1.

*"No se trata de un curso introductorio; es una inmersión profunda en la ingeniería de sistemas que actúan de manera autónoma en entornos complejos y dinámicos."*

## PRERREQUISITOS · DESEABLES (NO BLOQUEANTES)

### **Docker Básico**

El módulo de observabilidad (monitoreo en producción) usa un stack Loki + Tempo + Grafana en contenedores. Quien nunca corrió `docker compose up` va a perder tiempo específicamente ahí, sin que afecte el resto del curso.

### **Nociones de Arquitectura de Software**

Interfaces, modularidad, patrones de diseño. Ayuda mucho en los módulos de diseño de sistemas y orquestación multiagente (Módulos 2, 5 y 8), pero se explican en el curso a medida que se necesitan.

### **Haber Tocado Fine-Tuning o ML Supervisado, Aunque Sea Superficialmente**

El Módulo 7 (SFT / DPO / RLVR con TRL, PEFT y LoRA) es aplicado, no teórico — no se deriva el algoritmo matemático, pero si nunca se vio qué significa "entrenar" un modelo, es un salto conceptual grande en una sola clase.

Ninguno de estos tres bloquea el cursado — son fricciones puntuales, no barreras de entrada.





## QUÉ VAN A PODER HACER

**Construir agentes reales**

Desde el primer loop hasta el sistema en producción

**Diseñar arquitecturas**

Elegir el stack correcto para cada problema de negocio

**Medir y mejorar**

Eval, monitoreo, drift detection y ciclos de mejora continua

**Proteger sistemas**

MAESTRO, red teaming, LLM Guard, hardening de producción

**Liderar equipos**

Tomar decisiones de arquitectura y guiar adopción en organizaciones

**Dominar la terminología**

Frameworks, protocolos, métricas — el vocabulario de la industria

PRÓXIMOS PASOS

# Abrimos el diálogo:

## Consultas / Sugerencias

---

- 1 Revisá los pre-requisitos y confirmá que encaja con tu perfil
- 2 Contactá al equipo UTDT para fecha de inicio, modalidad y aranceles
- 3 El cupo por cohorte está dimensionado para sostener la calidad del curso

Universidad Torcuato Di Tella · Buenos Aires · 2026



# Escuela de Negocios Educación Ejecutiva